CS 544 (Spring 2026): Project 3 Report

Full Name: Ana Clara Toscano

Student ID Number: 9084692459

Don't change the number of pages, and don't let content for a specific part exceed the assigned page.

Part 2: Storage Performance

In your own words, what question are we asking with this experiment?

The question is which file format, zip or parquet, loads data faster. To compare this, we must measure the time it takes to read the file and convert it into memory. We run client.py with each format and record the execution time. The information of storage performance allows us to determine which file format loads the dataset into memory more quickly.

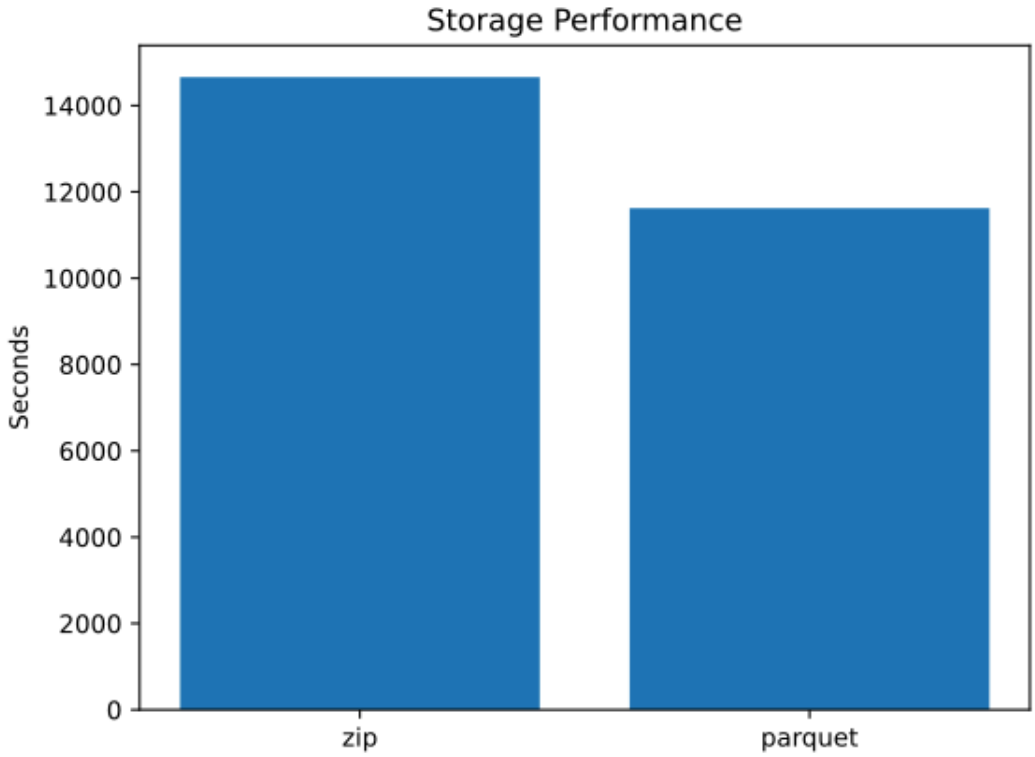


Figure 1: Storage performance comparison between zipped CSV and Parquet formats.

How do you interpret the results? Use the plot to answer the question you posed above. Be sure to reference at least one actual number in your specific plot as part of your answer.

The graph shows loading the zipped CSV took around 14,600 seconds, while loading the Parquet file took about 11,600 seconds. This evidence shows CSV is slower than Parquet. One reason behind this behavior is that CSV parsing requires converting strings into typed columns for every row when the file is read. In contrast, Parquet requires less parsing work during loading since it stores data in a columnar, pre-typed format, which results in faster performance when reading the dataset into memory.

Part 2: Memory Performance

In your own words, what question are we asking with this experiment?

The main question behind the experiment is does increasing the cache size improves the cache hit rate when processing many rows. Logically, when a cache is small, many requests miss the cache and there are more server calls. Similarly, when the cache is larger, more requests hit the cache resulting in a higher hit rate. This experiment tests how cache size affects the cache hit rate during client execution.

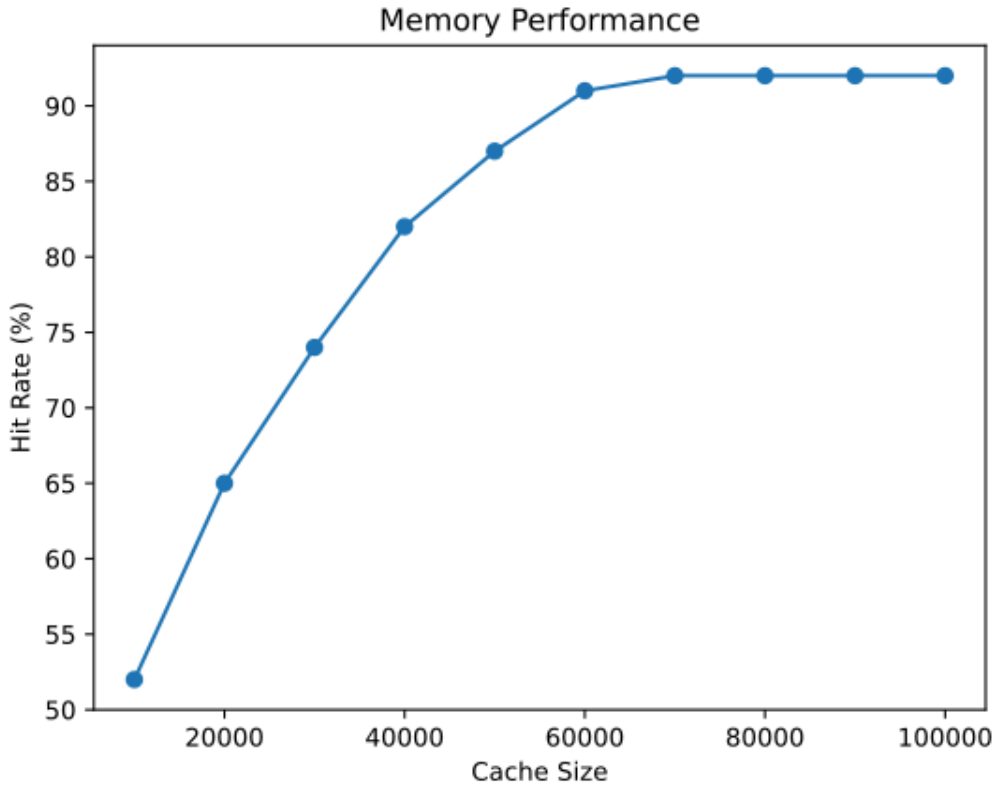


Figure 2: Memory hit rate percentage versus cache size.

How do you interpret the results? Use the plot to answer the question you posed above. Be sure to reference at least one actual number in your specific plot as part of your answer.

The plot shows that with a cache size of 10,000, the hit rate is about 52%. As the cache size increases, more previously requested values remain stored in memory, reducing the number of server calls and increasing the hit rate. The experiment showed that when the cache reaches 60,000, the hit rate rises to about 91%. These results show that larger cache sizes improve performance by increasing the hit rate. Note that the hit rate stabilizes around 92% for cache sizes between 70,000 and 100,000 because most frequently requested data is already stored in the cache for this dataset.

Part 4: Compute Performance

In your own words, what question are we asking with this experiment?

The experiment explores how the Global Interpreter Lock (GIL) affects performance as the number of threads increases. We measure impact on performance for two scenarios: GIL is enabled (threads take turns and cannot run in parallel), and GIL is disabled (threads can execute simultaneously on multiple CPU cores).

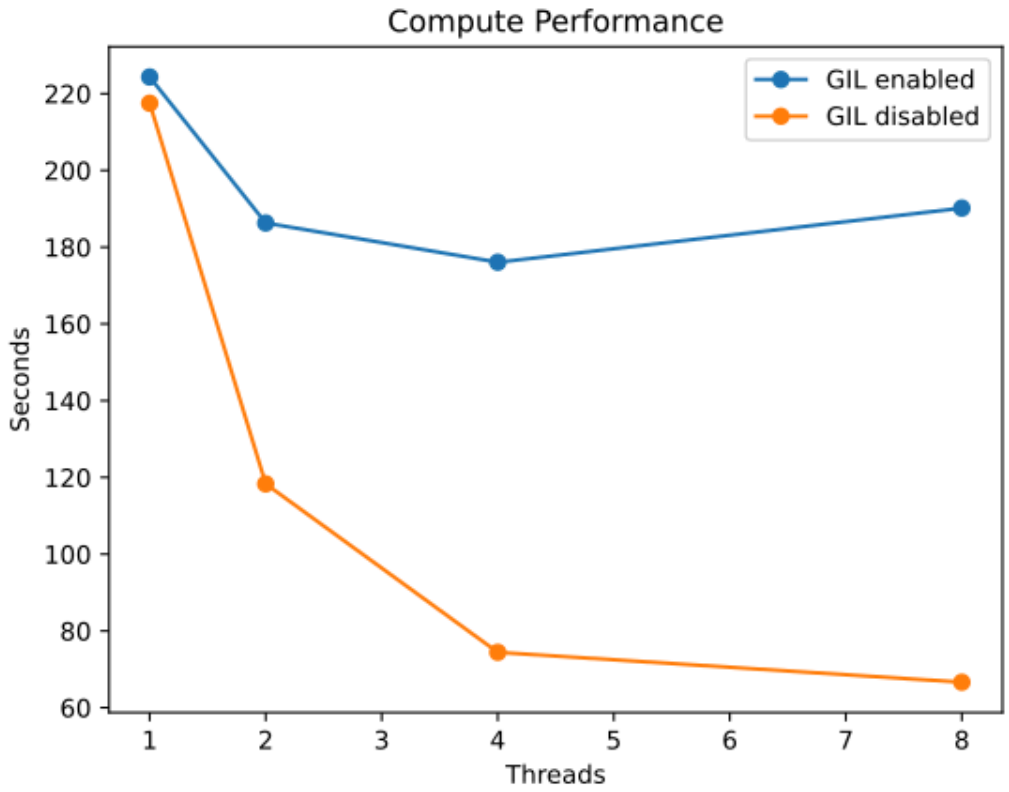


Figure 3: Compute performance comparison between GIL enabled and GIL disabled configurations.

How do you interpret the results? Use the plot to answer the question you posed above. Be sure to reference at least one actual number in your specific plot as part of your answer.

The plot shows how disabling the GIL improves compute performance as the number of threads increases. With GIL enabled, runtime decreases slightly from 224 seconds at 1 thread to 190 seconds at 8 threads, because threads take turns rather than running simultaneously. With GIL disabled, runtime drops from 218 seconds at 1 thread to 66 seconds at 8 threads, demonstrating efficient parallel execution on multiple CPU cores. This shows that the GIL restricts multi-threaded performance for CPU-bound tasks and that disabling it improves scalability.